

Slides: PAA_03

Disciplina: Projeto e Análise de Algoritmos.
Professor: André Tavares da Silva.
Aluno: Dalton Solano dos Reis.

Atividade 1

Escreva duas versões do algoritmo de fibonacci: a versão “ruim” apresentada e uma versão “boa” usando vetores. Faça uma comparação de tempo de execução com valores entre 25 e 60. Qual a complexidade da versão “boa” do algoritmo?

Resposta:

- Custo: tempo linear $O(n)$.
- Memória: linear $O(n)$.

fib(n) recursiva:

- Chama fib(n-1) + fib(n-2) até chegar nos casos base.
- É a versão mais parecida com a definição matemática.
- Repete muito trabalho. fib(30) recalcula vários mesmos valores muitas vezes.
- Custo: tempo exponencial, aproximadamente $O(2^n)$.
- Memória: $O(n)$ pela pilha de chamadas.
- No teste foi muito mais lenta: cerca de 0.16 s.

```
In [4]: import time

def fib(n: int) -> int:
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib(n - 1) + fib(n - 2)

inicio = time.perf_counter()
resultado = fib(30)
fim = time.perf_counter()

print("Fibonacci:", resultado)
print(f"Tempo com recursividade: {fim - inicio:.6f} segundos")

Fibonacci: 832040
Tempo com recursividade: 0.141105 segundos
```

fib_iterativo(n):

- Usa um laço e só guarda os dois últimos valores (a e b).
- Não repete contas.
- Custo: tempo linear $O(n)$.
- Memória: constante $O(1)$.
- É a melhor versão se o objetivo é só obter o valor final.
- No teste ficou em torno de 0.000028 s.

```
In [5]: import time

def fib_iterativo(n: int) -> int:
    if n == 0:
        return 0
    if n == 1:
        return 1

    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

inicio = time.perf_counter()
resultado = fib_iterativo(30)
fim = time.perf_counter()

print("Fibonacci sem recursividade:", resultado)
print(f"Tempo sem recursividade: {fim - inicio:.6f} segundos")

Fibonacci sem recursividade: 832040
Tempo sem recursividade: 0.000024 segundos
```

fib_vetor(n):

- Também é linear, mas guarda todos os resultados anteriores em uma lista fibs.
- Custo: tempo linear $O(n)$.
- Memória: linear $O(n)$.
- É útil quando você quer acessar vários Fibonacci já calculados, não só o último.
- Teste ficou quase igual à iterativa, mas usa mais memória.

```
In [6]: import time

def fib_vetor(n: int) -> int:
    if n == 0:
        return 0
    if n == 1:
        return 1

    fibs = [0] * (n + 1)
    fibs[0] = 0
    fibs[1] = 1

    for i in range(2, n + 1):
        fibs[i] = fibs[i - 1] + fibs[i - 2]

    return fibs[n]

inicio = time.perf_counter()
resultado = fib_vetor(30)
fim = time.perf_counter()

print("Fibonacci com vetor:", resultado)
print(f"Tempo com vetor: {fim - inicio:.6f} segundos")

Fibonacci com vetor: 832040
Tempo com vetor: 0.000027 segundos
```